

KLİNİK VERİ SINIFLANDIRMA SİSTEMİ: KURAL TABANLI HASTA KAYIT YÖNETİMİ

Abdulmutin

Kastamonu Üniversitesi, Bilgisayar Mühendisliği Bölümü, 2. Sınıf
Veri Tabanı Yönetim Sistemleri Dersi Proje Raporu

Özet- Bu rapor, ASP.NET Core MVC, Entity Framework Core ve PostgreSQL kullanılarak geliştirilen Klinik Veri Sınıflandırma Sistemi projesini açıklamaktadır. Proje, hasta bilgilerini, hastalara ait klinik kayıtları ve bu kayıtlara bağlı sınıflandırma kategorilerini ilişkisel veritabanı üzerinde yönetir. Klinik kayıtlar oluşturulurken veya güncellenirken nabız, tansiyon, kan şekeri, kolesterol, vücut sıcaklığı, tanı ve not alanları kural tabanlı bir servis tarafından değerlendirilir. Sonuç olarak kayıtlar Normal, Riskli, Acil veya Kronik Takip Gerekli sınıflarından birine atanır. Uygulama; veri tabanı tasarımı, foreign key ilişkileri, Code First migration, seed data, CRUD işlemleri, filtreleme ve dashboard istatistikleri açısından Veri Tabanı Yönetim Sistemleri dersi için uygun bir örnek oluşturmaktadır.

Anahtar Kelimeler- PostgreSQL, Entity Framework Core, ASP.NET Core MVC, Klinik Veri, Sınıflandırma, Veri Tabanı Yönetim Sistemleri.

I. GİRİŞ

Sağlık alanında hasta ve klinik kayıtlarının düzenli tutulması, veriye dayalı karar alma süreçlerinin temelini oluşturur. Hasta kimlik bilgileri, muayene kayıtları, ölçüm değerleri ve risk sınıfları birbirleriyle ilişkili verilerden oluşur. Bu nedenle klinik kayıt yönetimi için ilişkisel veritabanı tasarımı uygun bir yaklaşımdır.

Klinik Veri Sınıflandırma Sistemi, Veri Tabanı Yönetim Sistemleri dersi kapsamında hazırlanmış akademik bir web uygulamasıdır. Proje, hasta bilgilerini ve klinik ölçümleri saklamanın yanında, girilen klinik kaydı otomatik olarak sınıflandırır. Böylece veritabanı işlemleri yalnızca kayıt ekleme ve listeleme ile sınırlı kalmaz; kayıtların anlamlı kategorilere ayrılması da sağlanır.

II. AMAÇ VE KAPSAM

Projenin temel amacı, hasta ve klinik kayıtları için ilişkisel bir veritabanı modeli kurmak ve bu model üzerinde tam CRUD işlemleri gerçekleştirmektir. Ayrıca klinik değerleri belirli eşiklere göre değerlendirerek otomatik sınıflandırma yapmak hedeflenmiştir.

Proje kapsamındaki ana hedefler şunlardır:

- Hasta bilgilerini benzersiz hasta numarası ile saklamak.
- Bir hastaya birden fazla klinik kayıt bağlamak.
- Klinik kayıtları otomatik olarak sınıflandırmak.
- Sınıflandırma kategorilerini veritabanında yönetilebilir hale getirmek.
- Kayıtları sınıf, risk seviyesi ve tarih aralığına göre filtrelemek.
- Dashboard ekranında toplam ve sınıf bazlı istatistikler sunmak.

III. KULLANILAN TEKNOLOJİLER

Uygulama .NET 8 üzerinde çalışan bir ASP.NET Core MVC projesidir. Veritabanı işlemlerinde Entity Framework Core kullanılmıştır. PostgreSQL ilişkisel veritabanı olarak seçilmiş, bağlantı Npgsql sağlayıcısı ile kurulmuştur. Arayüz Razor View ve Bootstrap 5 ile hazırlanmıştır. Veritabanı ortamı Docker Compose ile yerel olarak çalıştırılabilir.

Tablo I. Kullanılan Teknolojiler

Teknoloji	Görevi
ASP.NET Core MVC	Controller, Model ve View tabanlı web mimarisi
Entity Framework Core	ORM, Code First migration ve veri erişimi
PostgreSQL	Hasta, klinik kayıt ve sınıf verilerinin saklanması
Npgsql	.NET uygulaması ile PostgreSQL bağlantısı
Bootstrap 5	Responsive ve sade kullanıcı arayüzü
Docker Compose	PostgreSQL servisinin kolay çalıştırılması

IV. SİSTEM MİMARİSİ

Proje MVC mimarisi ile katmanlara ayrılmıştır. Controller sınıfları HTTP isteklerini karşılar ve veritabanı işlemleri için ApplicationDbContext kullanır. Klinik kayıt oluşturma ve güncelleme sırasında sınıflandırma mantığı doğrudan controller içine yazılmamış, IClassificationService arayüzü ve ClassificationService sınıfı üzerinden ayrılmıştır. Bu yapı hem okunabilirliği artırır hem de iş kuralını merkezi hale getirir.

Tablo II. Klasörler ve Sorumluluklar

Katman	Örnek Dosyalar	Sorumluluk
Models	Patient, ClinicalRecord, ClinicalClass	Veritabanı varlıkları ve doğrulama kuralları
Data	ApplicationDbContext, SeedData	DbSet tanımları, ilişkiler, seed data
Services	ClassificationService	Kural tabanlı otomatik sınıflandırma
Controllers	PatientsController, ClinicalRecordsController	CRUD ve sayfa akışları
Views	Razor .cshtml dosyaları	Kullanıcı arayüzü
Migrations	Migration dosyaları	Veritabanı şemasının sürümlemesi

V. VERİTABANI TASARIMI

Veritabanı üç ana tablo üzerine kurulmuştur: Patients, ClinicalRecords ve ClinicalClasses. Patients tablosu hastanın ad, soyad, hasta numarası, doğum tarihi, cinsiyet, telefon ve adres bilgilerini tutar. ClinicalRecords tablosu hastaya ait şikayet, tanı, tansiyon, nabız, vücut sıcaklığı, kan şekeri, kolesterol, not ve kayıt tarihi alanlarını içerir. ClinicalClasses tablosu ise Normal, Riskli, Acil ve Kronik Takip Gerekli gibi sınıfları saklar.

Bir hasta birden fazla klinik kayda sahip olabilir. Bu nedenle Patients ile ClinicalRecords arasında bire çok ilişki kurulmuştur. Her klinik kayıt bir sınıflandırma kategorisine bağlıdır. Bu ilişki ClinicalRecords.ClinicalClassId foreign key alanı ile sağlanır. Klinik sınıf silme davranışında DeleteBehavior.Restrict kullanılarak bağlı kayıt bulunan sınıfların yanlışlıkla silinmesi engellenmiştir.

Tablo III. Veritabanı Tabloları

Tablo	Ana Alanlar	Açıklama
Patients	Id, FirstName, LastName, PatientNumber	Hasta temel bilgileri
ClinicalRecords	PatientId, ClinicalClassId, Diagnosis, Pulse, BloodSugar	Hastaya bağlı klinik ölçüm ve tanı kayıtları
ClinicalClasses	Name, RiskLevel, ColorCode	Sınıflandırma kategorileri ve risk seviyeleri

VI. VERİ BÜTÜNLÜĞÜ VE DOĞRULAMA

PatientNumber alanı için unique index tanımlanmıştır. Böylece aynı hasta numarası ile birden fazla hasta kaydı oluşturulması engellenir. ClinicalClass.Name alanı da unique index ile korunur. Model sınıflarında Required, StringLength, Range, Phone ve DataType gibi data annotation öznitelikleri kullanılmıştır. Bu öznitelikler hem kullanıcı arayüzünde hem de model doğrulama sürecinde veri kalitesini artırır.

Sayısal klinik değerlerde negatif değer girişinin önüne geçmek için aralık kontrolleri bulunmaktadır. Örneğin nabız 0-250, vücut sıcaklığı 0-45, kan şekeri ve kolesterol 0-1000 aralığında doğrulanır.

VII. SINIFLANDIRMA ALGORİTMASI

Sınıflandırma işlemi ClassificationService içinde yer alır. Klinik kayıt oluşturulurken veya güncellenirken servis çağrılır ve uygun sınıf adı döndürülür. Controller bu sınıf adına karşılık gelen ClinicalClass kaydını veritabanından bulur ve ClinicalClassId alanına yazar.

Algoritma önce nabız kontrol eder. Nabız 50'nin altında veya 120'nin üzerindeyse kayıt doğrudan Acil olarak sınıflandırılır. Daha sonra kan şekeri, vücut sıcaklığı, tansiyon ve kolesterol değerleri için anormal değer sayısı hesaplanır.

$$\text{Anormal Sayısı} = \text{Kan Şekeri} + \text{Ateş} + \text{Tansiyon} + \text{Kolesterol göstergeleri}$$

Tablo IV. Sınıflandırma Kuralları

Koşul	Sonuç
Nabız < 50 veya > 120	Acil
Kan şekeri > 180	Anormal değer sayısı artar
Vücut sıcaklığı > 38	Anormal değer sayısı artar
Büyük tansiyon >= 140 veya küçük tansiyon >= 90	Anormal değer sayısı artar
Kolesterol > 240	Anormal değer sayısı artar
Anormal değer sayısı >= 2	Acil
Tanı veya notlarda kronik/takip geçmesi	Kronik Takip Gerekli
Tek anormal değer	Riskli
Anormal değer yok	Normal

VIII. UYGULAMA MODÜLLERİ

Dashboard modülü toplam hasta sayısını, toplam klinik kayıt sayısını, Normal, Riskli ve Acil sınıflarındaki kayıt sayılarını ve son klinik kayıtları gösterir. Ayrıca sınıf istatistikleri kategori bazında listelenir.

Hasta Yönetimi modülü hasta listeleme, arama, ekleme, güncelleme, detay görüntüleme ve silme işlemlerini içerir. Arama işlemi ad, soyad ve hasta numarası üzerinden yapılabilir. Klinik Kayıt Yönetimi modülü klinik kayıt CRUD işlemlerini, otomatik sınıflandırmayı ve sınıf/risk/tarih filtrelerini sağlar. Sınıflandırma Kategorileri modülü kategori ekleme, düzenleme, detay ve silme işlemlerini içerir; bağlı klinik kayıt varsa kategori silinemez.

IX. SEED DATA VE MİGRATION

Projede Code First yaklaşımı kullanılmıştır. Migration dosyaları veritabanı şemasını sürümlendirir. `ApplicationDbContext.OnModelCreating` içinde varsayılan sınıflandırma kategorileri, örnek hasta kayıtları ve örnek klinik kayıtlar `HasData` ile tanımlanmıştır. Ayrıca `SeedData.cs` dosyasında runtime seed örneği bulunmaktadır.

Varsayılan sınıflar Normal, Riskli, Acil ve Kronik Takip Gerekli olarak belirlenmiştir. Örnek kayıtlarda rutin kontrol, hipertansiyon şüphesi ve kronik diyabet takibi gibi senaryolar bulunur. Bu seed veriler, uygulama ilk çalıştırıldığında ekranların dolu ve test edilebilir olmasına yardımcı olur.

X. ÖRNEK SENARYO

Bir kullanıcı Mehmet Demir adlı hasta için baş ağrısı ve halsizlik şikayetiyle klinik kayıt oluşturduğunda, sistem tansiyon değerlerini ve diğer ölçümleri değerlendirir.

Büyük tansiyon 152 ve küçük tansiyon 96 olduğu için tansiyon göstergesi anormal kabul edilir. Eğer birden fazla anormal değer oluşursa kayıt Acil sınıfına atanır; tek anormal değer varsa Riskli sınıfına atanır. Tanı veya not alanında kronik ya da takip ifadesi geçerse kayıt Kronik Takip Gerekli sınıfına yönlendirilir.

XI. SONUÇ

Klinik Veri Sınıflandırma Sistemi, Veri Tabanı Yönetim Sistemleri dersi için ilişkisel veritabanı tasarımı, veri bütünlüğü, migration, seed data ve CRUD işlemlerini uygulamalı biçimde gösteren bir projedir. ASP.NET Core MVC ve Entity Framework Core ile geliştirilen sistem, PostgreSQL üzerinde hasta ve klinik kayıtlarını yönetir. Kural tabanlı sınıflandırma servisi sayesinde klinik veriler yalnızca saklanmakla kalmaz, anlamlı risk kategorilerine ayrılır. Proje ileride kullanıcı yetkilendirme, gelişmiş raporlama, grafikler ve daha ayrıntılı klinik karar kuralları ile genişletilebilir.

KAYNAKÇA

- [1] Microsoft, "ASP.NET Core MVC documentation," Microsoft Learn.
- [2] Microsoft, "Entity Framework Core documentation," Microsoft Learn.
- [3] PostgreSQL Global Development Group, "PostgreSQL Documentation."
- [4] Klinik Veri Sınıflandırma Sistemi proje kaynak kodları ve README dosyası.